

## Data store API

```
// accessor for the Datastore service
private final Datastore datastore = DatastoreOptions.getDefaultInstance().getService();

// ...

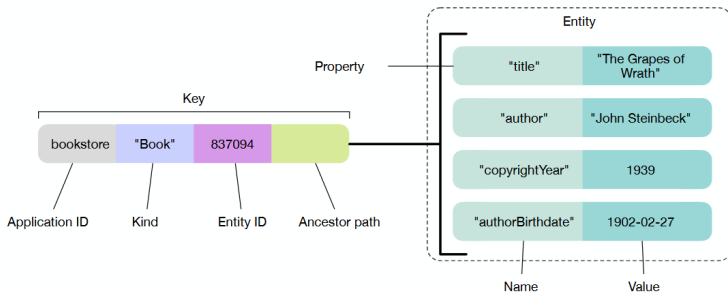
KeyFactory keyFactory = datastore.newKeyFactory().setKind("book");
Key key = datastore.allocateId(keyFactory.newKey());
Date authorBirthdate = new GregorianCalendar(1902, GregorianCalendar.FEBRUARY, 27).getTime();
Entity entity = Entity.newBuilder(key)
    .set("title", "The grapes of wrath")
    .set("author", "John Steinbeck")
    .set("copyrightYear", 1939)
    .set("authorBirthdate", Timestamp.of(authorBirthdate))
    .build();
datastore.put(entity);
```

### Class declaration

```
@Entity(name = "Book")
public class Book {
    @Id
    private String isbn;
    private String title;
    private String author;
    private int copyrightYear;
    private Date authorBirthdate;
    // ... constructors, accessors
}
```

### Store class declaration

```
EntityManager em = null;
try {
    em = emf.createEntityManager();
    Book book = new Book();
    book.setTitle("The Grapes of Wrath");
    // ...
    em.persist(book);
} finally {
    if (em != null)
        em.close();
}
```



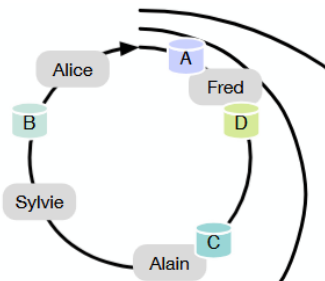
## Horizontal scaling

### Replication (single head) : just copy the entier thing else were

- one instance is the main one (leader) and the other replicate the main one (follower)
- To increase robustness, leader and followers should be in different availability zones.
- Database slow down a bit with synchronous replication

### Partition : split the data base across a multitude of machin

- En gros, a chaque requet faut passé par un "request router" qui vas rediriger la requet ver la bonne instance
- Les request router sont rapide et ne communique pas entre eux
- une hash fonction est utiliser pour determiner sur qu'elle instance vas chaque donner pour que se sois rapide a chercher (comme les bucket d'une hashmap)



### Document model

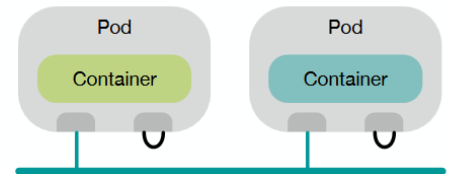
un peut style JSON si on veut

### Graph model

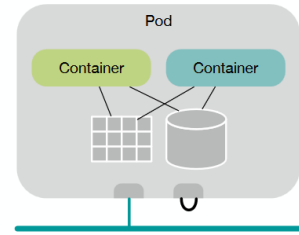
Cool pour les relation  
en gros, bah tu fais un graph d'objet



### Loosely-coupled containers



### Tightly-coupled containers



## Cloud database (Database-as-a-Service)

En gros, c'est une database géré par le fournisseur cloud

### Single-tenant cloud databases

the database is dedicated to a single cloud client

basé souvent sur une techno existante

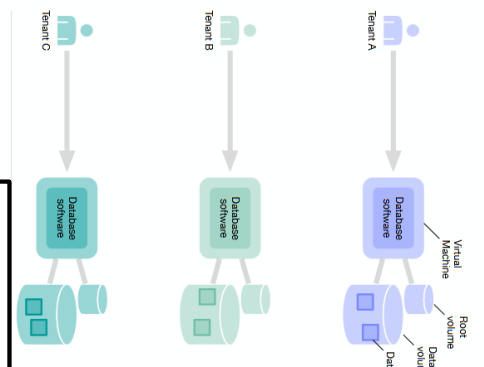
quand on a besoin d'un nouveaux volume, on crée un VM

### Multi-tenant database

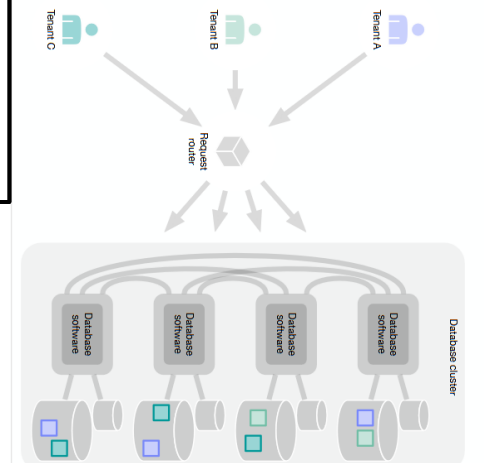
the database is shared between several cloud clients.

utilise souvent du NoSQL

Y a une database distribuer a travers plein d'instance



Single-tenant databases



Multi-tenant database

## YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: www
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - mountPath: /srv/www
      name: www-data
      readOnly: true
  - name: git-monitor
    image: kubernetes/git-monitor
    env:
    - name: GIT_REPO
      value: http://github.com/some/repo.git
    volumeMounts:
    - mountPath: /data
      name: www-data
  volumes:
  - name: www-data
    emptyDir: {}
```

-Name of Pod

Internal name of the container  
in the Pod

- Container image name

Environment variable name  
and value for the container

```
(docker run -e  
GIT_REPO=... ..)
```

- Volume name and type

## Object

un objet est composé dans l'ordre de :

- api version
- kind
- d'une struct metadata (info de l'objet)
- d'une struct spec (targeted state de l'objet)

Quand Kurb crée un objet, il ajoute des donnée sur sont yaml :

dans le metadata

- uid de l'objet
- ressourceVersion de l'objet

et aussi ajoute la struct **status** qui à des info sur le state actuel de l'objet (en read only)

## Terraform

C'est un outil qui permet faire du laC opensource

## Concept

## Execution plan

Terraform generates an execution plan describing what it will do to reach the desired state

## Resource graph

Terraform builds a graph of all resources, and parallelizes the creation and modification of any non-dependent resources

## Play 1

## Play 2

### Play 3

```
- name: Base layer common to puppy and kitty node
hosts: all
tasks:
- name: Ensure NGINX is installed.
  apt:
    name: nginx
    state: present
    update_cache: yes

- name: Configuration specific to puppy node
hosts: puppy
tasks:
- name: Ensure puppy group is present.
  group:
    name: puppy
    state: present
...

- name: Configuration specific to kitty node
hosts: kitty
tasks:
- name: Ensure kitty group is present.
  group:
    name: kitty
    state: present
...
```

## Type de volume

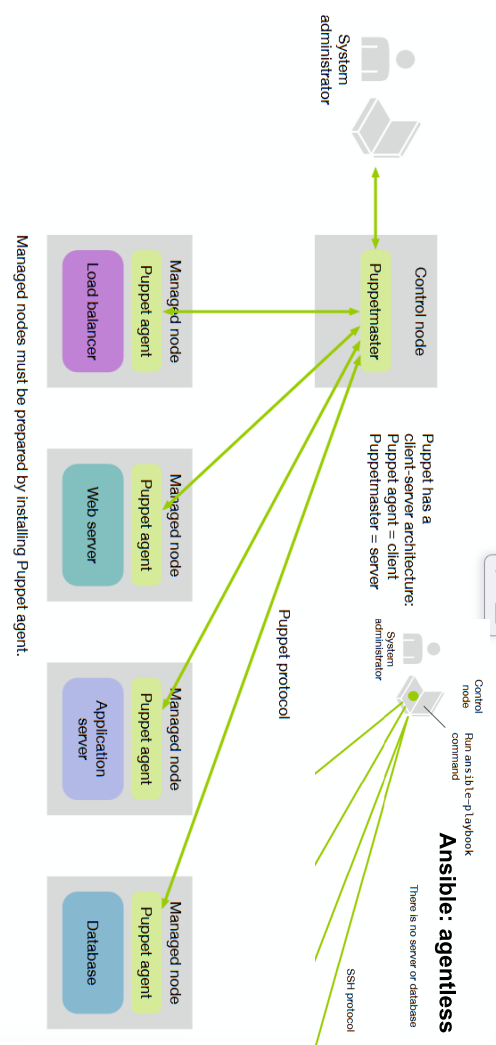
- emptyDir : empty volume | del with pod
- hostPath : path sur la machin host | persite
- secret : volume crypter
- NFS : network file system | persite

laas

- Launch a virtual machine.
- Connect to the VM via SSH.
- Install application on VM.
- Create VM image from VM.
- Create load balancer.
- To launch more VMs from VM image create a Launch Template and an AutoScaling Group.
- Connect VMs to load balancer.

## Kubernetes

- On local machine, create container image for app using Docker and Dockerfile.
- Upload container image to a container registry.
- Write a Kubernetes **manifest** file in YAML that describes a **Pod** that will contain your container.
- Launch the Pod (with the container) using the command line:  
kubectl create -f pod.yaml
- To launch multiple replicas of your container, write a **Deployment** manifest.
- For load balancing, write a **Service** manifest.



### Desired-State Configuration (déclarative au lieu d'imperative)

## Idempotent commands

exemple : ssh-keygen, 1st run = create new key, 2nd run = overwrite old key with new ones